

实验五 哈夫曼树的设计及实现

一、实验目的

1. 掌握哈夫曼树的构造算法，理解二叉树的应用；
2. 掌握哈夫曼编码的构造算法。

二、实验内容

输入一串字符串，根据给定的字符串中字符出现的频率建立相应的哈夫曼树，构造哈夫曼编码表，在此基础上可以对压缩文件进行压缩（即编码）。

已知字符串中出现的字符为 A、B、C、D、E、F、G、H，其相应的权值为

7、19、2、6、32、3、21、10。

```
//主函数
int main()
{
    int n = 0;
    printf("请输入数据个数:>");
    scanf("%d", &n);
    DataType* w = (DataType*)malloc(sizeof(DataType)*n);
    if (w == NULL)
    {
        printf("malloc fail\n");
        exit(-1);
    }
    printf("请输入数据:>");
    for (int i = 0; i < n; i++)
    {
        scanf("%lf", &w[i]);
    }
    HuffmanTree HT;
    CreateHuff(HT, w, n); //构建哈夫曼树

    HuffmanCode HC;
    HuffCoding(HT, HC, n); //构建哈夫曼编码

    for (int i = 1; i <= n; i++) //打印哈夫曼编码
    {
        printf("数据%.2lf的编码为:%s\n", HT[i].weight, HC[i]);
    }
    free(w);
    return 0;
}
```

```

16 //在下标为1到i-1的范围找到权值最小的两个值的下标，其中s1的权值小于s2的权值
17 void Select(HuffmanTree& HT, int n, int& s1, int& s2)

```

```

18 {
19     int min;
20     //找第一个最小值
21     for (int i = 1; i <= n; i++)
22     {
23         if (HT[i].parent == 0)
24         {
25             min = i;
26             break;
27         }
28     }
29     for (int i = min + 1; i <= n; i++)
30     {
31         if (HT[i].parent == 0 && HT[i].weight < HT[min].weight)
32             min = i;
33     }
34     s1 = min; //第一个最小值给s1
35     //找第二个最小值
36     for (int i = 1; i <= n; i++)
37     {
38         if (HT[i].parent == 0 && i != s1)
39         {
40             min = i;
41             break;
42         }
43     }
44     for (int i = min + 1; i <= n; i++)
45     {
46         if (HT[i].parent == 0 && HT[i].weight < HT[min].weight && i != s1)
47             min = i;
48     }
49     s2 = min; //第二个最小值给s2
50 }

```

```

52 //构建哈夫曼树

```

```

53 void CreateHuff(HuffmanTree& HT, DataType* w, int n)
54 {
55     int m = 2 * n - 1; //哈夫曼树总结点数
56     HT = (HuffmanTree) calloc(m + 1, sizeof(HTNode)); //开m+1个HTNode，因为下标为0的HTNode不存储
57     for (int i = 1; i <= n; i++)
58     {
59         HT[i].weight = w[i - 1]; //赋权值给n个叶子结点
60     }
61     for (int i = n + 1; i <= m; i++) //构建哈夫曼树
62     {
63         //选择权值最小的s1和s2，生成它们的父结点
64         int s1, s2;
65         Select(HT, i - 1, s1, s2); //在下标为1到i-1的范围找到权值最小的两个值的下标，其中s1的权值小于
66         HT[i].weight = HT[s1].weight + HT[s2].weight; //i的权重是s1和s2的权重之和
67         HT[s1].parent = i; //s1的父亲是i
68         HT[s2].parent = i; //s2的父亲是i
69         HT[i].lc = s1; //左孩子是s1
70         HT[i].rc = s2; //右孩子是s2
71     }
72     //打印哈夫曼树中各结点之间的关系
73     printf("哈夫曼树为:>\n");
74     printf("下标    权值    父结点    左孩子    右孩子\n");
75     printf("0\n");
76     for (int i = 1; i <= m; i++)
77     {
78         printf("%-4d    %-6.2lf    %-6d    %-6d    %-6d\n", i, HT[i].weight, HT[i].parent, HT[i].lc, HT[i].rc);
79     }
80     printf("\n");
81 }
82

```

```

83 //生成哈夫曼编码
84 void HuffCoding(HuffmanTree& HT, HuffmanCode& HC, int n)
85 {
86     HC = (HuffmanCode)malloc(sizeof(char)*(n + 1)); //开n+1个空间, 因为下标为0的空间不用
87     char* code = (char*)malloc(sizeof(char)*n); //辅助空间, 编码最长为n(最长时, 前n-1个用于存储数据)
88     code[n - 1] = '\0'; //辅助空间最后一个位置为'\0'
89     for (int i = 1; i <= n; i++)
90     {
91         int start = n - 1; //每次生成数据的哈夫曼编码之前, 先将start指针指向'\0'
92         int c = i; //正在进行的第i个数据的编码
93         int p = HT[c].parent; //找到该数据的父结点
94         while (p) //直到父结点为0, 即父结点为根结点时, 停止
95         {
96             if (HT[p].lc == c) //如果该结点是其父结点的左孩子, 则编码为0, 否则为1
97                 code[--start] = '0';
98             else
99                 code[--start] = '1';
100             c = p; //继续往上进行编码
101             p = HT[c].parent; //c的父结点
102         }
103         HC[i] = (char*)malloc(sizeof(char)*(n - start)); //开辟用于存储编码的内存空间
104         strcpy(HC[i], &code[start]); //将编码拷贝到字符指针数组中的相应位置
105     }
106     free(code); //释放辅助空间
107 }

```

0 Quiz && /Users/cyyao/Desktop/C/C++/ Quiz

请输入数据个数:>8

请输入数据:>7 19 2 6 32 3 21 10

哈夫曼树为:>

下标	权值	父结点	左孩子	右孩子
0				
1	7.00	11	0	0
2	19.00	13	0	0
3	2.00	9	0	0
4	6.00	10	0	0
5	32.00	14	0	0
6	3.00	9	0	0
7	21.00	13	0	0
8	10.00	11	0	0
9	5.00	10	3	6
10	11.00	12	9	4
11	17.00	12	1	8
12	28.00	14	10	11
13	40.00	15	2	7
14	60.00	15	12	5
15	100.00	0	13	14

数据7.00的编码为:1010

数据19.00的编码为:00

数据2.00的编码为:10000

数据6.00的编码为:1001

数据32.00的编码为:11

数据3.00的编码为:10001

数据21.00的编码为:01

数据10.00的编码为:1011